

Chapter 57. LLMOps

Prompts as code, golden sets in CI, surviving a provider's model change, metering, caching, fallbacks, and monitoring without ground truth.

Draft chapter

Contents

Chapter 57. LLMOps

Why this matters

In plain English

57.1 What changes, and what doesn't

57.2 Prompts as code

57.3 The golden set as a build step

57.4 The day the provider changes the model

57.5 Tokens, cost, and the meter

57.6 Caching

57.7 Latency

57.8 Failure and fallback

57.9 Monitoring without ground truth

57.10 Safety and logs in production

57.11 The human loop

Common mistakes and how to spot them

In the real world: the Monday the orders stopped

Tools

The project: instrument an LLM feature

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 57. LLMOps

Part VI — Production ML, Generative AI & MLOps

Chapter at a glance

You will learn to: name what LLMOps adds to Chapter 56's discipline, and why the model you depend on isn't yours · treat prompts as versioned code with a score attached · run a golden set in CI with a floor that fails the build · survive the day a provider ships a new model behind the same name · meter tokens and project a monthly bill · cache, and measure what it saves · budget latency · retry, fall back, and degrade honestly when the provider fails · monitor a system that has no accuracy to monitor · keep logs that are useful and lawful · and run the human review loop that turns complaints into test cases.

Before you start: Chapter 54 (prompting, structured output, token costs), Chapter 55 (the assistant, golden sets, guardrails), Chapter 56 (versioning, serving, monitoring, retraining, incidents), Chapter 29 (retries, exit codes).

Time needed: 14–18 hours, spread over three weeks.

Tools: Python 3.12, NumPy, scikit-learn. The provider is Chapter 54's local stand-in, extended so a version change and a failure rate can be simulated offline.

Practice systems: Chapter 54's order-extraction pipeline (60 emails with ground truth) and Chapter 55's support assistant, plus eight weeks of incoming questions built by `questions_stream.py`.

Why this matters

Chapter 56 kept a model alive that you trained, on data you own, with an accuracy you can measure. An LLM feature breaks all three of those assumptions:

- **You don't own the model.** It can change behind the same name, be deprecated with 60 days' notice, or rate-limit you at the worst moment.
- **There is no accuracy.** The output is text. For Riverstone's extraction pipeline you can build ground truth; for the support assistant, "was that a good answer?" is a judgment, and judgment doesn't come in a dashboard.
- **The cost is per request, forever.** A model you host has a fixed bill; a model you call has one that scales with success.

The discipline that answers this is the same shape as Chapter 56 and a different set of controls. It is also, right now, the difference between the AI features that are still running a year after the demo and the ones that were quietly switched off.

In plain English

You have built a business process on somebody else's engine, and you can't open the bonnet.

That's not a reason to avoid it. It is a reason to build four things around it:

- **A test you trust.** A set of inputs with known right answers, run on every change, including changes you didn't make. It is the only way to notice that the engine was swapped.
- **A meter.** Tokens in, tokens out, per request, so the bill is a number you predicted rather than one that arrives.
- **A spare.** Retries, a cheaper model, and a degraded mode that still serves customers when the provider is down.
- **A way to hear complaints.** With no accuracy to measure, the fastest signal is the person who says "that answer was wrong", and the job is to make that cheap to say and impossible to lose.

The rest is Chapter 56's discipline with two extra things versioned: **the prompt**, and **the model you don't control**.

57.1 What changes, and what doesn't

Chapter 56 said version...	In an LLM system that is...
Data	the corpus and the index (Chapter 55), plus the golden set
Features	the chunking and retrieval configuration
Code	the application, the parser, the validators
Model artifact	a model name and version string you do not control
Config	temperature, max tokens, thresholds, and the prompt

Two additions, and both are the source of most LLMOps incidents:

- **The prompt is config that behaves like code.** It changes the system's behavior, it can regress, and it needs a version, a review, and a test.
 - **The model is a dependency with its own release schedule.** Treat it exactly as you treat a library you pin: name the version, test before upgrading, and keep the old one available while you can.
-

57.2 Prompts as code

Riverstone's extraction prompts live in one module, each with a version:

```
import sys
sys.path.insert(0, ".")
from pipeline import PROMPTS, evaluate

for version in ("v1", "v2", "v3"):
    result = evaluate(version)
    print(f"prompt {version}: {result['exact']}>2} of {result['of']} correct "
          f"({result['accuracy']:.0%}), {result['unparseable']} unparseable")
```

```
prompt v1: 21 of 60 correct (35%), 4 unparseable
prompt v2: 35 of 60 correct (58%), 0 unparseable
prompt v3: 47 of 60 correct (78%), 0 unparseable
```

Line by line: `PROMPTS` is a dictionary of version string to template, `evaluate` runs the whole golden set and returns the counts. Every prompt version has a number beside it, permanently, which is the only way to answer "was last month's version better?".

What a prompt registry needs, whether it is a dictionary, a YAML file, or a product:

Field	Why
Version	so a rollback names something
The text itself, in Git	so changes are reviewed and diffable
Model and settings it was tested with	a prompt is only good <i>with</i> a model at a temperature
Golden-set score	the reason to prefer it
Date and author	the incident question is always "what changed and when?"
Status: draft, live, retired	so nobody guesses which one production uses

Watch out: a prompt change is a release, not an edit. The most common LLMOps incident is somebody improving a prompt in a dashboard on Friday afternoon. Same review, same test, same rollback plan as code, because it is code: it changes what the system does to customers.

57.3 The golden set as a build step

Chapter 55 built the evaluation set; here it becomes a gate.

```
FL00R = 0.70

result = evaluate("v3")
status = "PASS" if result["accuracy"] >= FL00R and result["unparseable"] == 0 else "FAIL"
print(f"golden set: {result['exact']} of {result['of']} ({result['accuracy']:.0%}), "
      f"unparseable {result['unparseable']}, floor {FL00R:.0%} -> {status}")
print("a build script would exit 0 on PASS and 1 on FAIL (Chapter 29's exit codes)")
```

```
golden set: 47 of 60 (78%), unparseable 0, floor 70% -> PASS
a build script would exit 0 on PASS and 1 on FAIL (Chapter 29's exit codes)
```

Line by line: two conditions, not one. Accuracy above a floor is obvious; `unparseable == 0` is the condition that actually saves you, because a formatting change breaks every record at once while accuracy metrics computed on the parseable subset can look fine.

What it costs to run. One pass over the 60-email golden set is about 11,000 input and 3,200 output tokens: at Chapter 54's workhorse prices, roughly ₹4.74, or under a rupee for a ten-email smoke test. Running it on every commit and nightly against the live provider is affordable at any volume a mid-sized company has, and the nightly run is the one that catches the next section's problem.

57.4 The day the provider changes the model

Providers ship improvements behind the same model name. Usually nothing happens. Occasionally your parser stops working, and the first you hear of it is from a customer.

The companion `provider.py` simulates one: same task, slightly different habits. It comments its answers, and on some emails it reverts to the date format the email itself used.

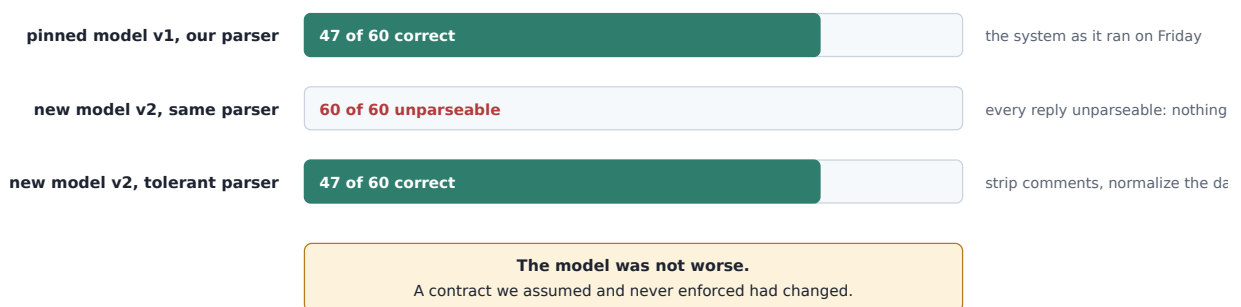
```
old_parser = evaluate("v3", model="v2", tolerant=False)
print(f"new model version, existing parser: {old_parser['exact']} of {old_parser['of']} correct, "
      f"{old_parser['unparseable']} unparseable")

tolerant = evaluate("v3", model="v2", tolerant=True)
print(f"new model version, tolerant parser: {tolerant['exact']} of {tolerant['of']} correct, "
      f"{tolerant['unparseable']} unparseable")

baseline = evaluate("v3", model="v1")
print(f"for comparison, the pinned model: {baseline['exact']} of {baseline['of']} correct")
```

```
new model version, existing parser: 0 of 60 correct, 60 unparseable
new model version, tolerant parser: 47 of 60 correct, 0 unparseable
for comparison, the pinned model: 47 of 60 correct
```

The day the provider changed the model behind the same name



Nightly golden-set runs turn this into a Saturday-night build failure. Without them, it is a Monday morning with no orders in the ERP.

Figure 57.1 — A provider update nobody triggered, and the twenty-minute fix that was always in your own code.

Read that first line again: zero of sixty. A provider update that nobody at Riverstone triggered took a pipeline from 78% to nothing, silently, because two habit changes made every reply unparseable. Without a nightly golden-set run, this is discovered when the ERP has no orders in it on Monday

morning.

The fix is not a better prompt. It is defensive parsing in *your* code:

```
def parse(reply, tolerant=True):
    """Get JSON out of whatever came back. Tolerant parsing is cheap insurance against a model's
    habits."""
    text = reply.split("```json")[-1].split("```")[0] if "```" in reply else reply
    if tolerant:
        text = "\n".join(line for line in text.splitlines() if not line.strip().startswith("//"))
    try:
        order = json.loads(text)
    except json.JSONDecodeError:
        return None
    if tolerant:
        order["delivery_date"] = normalize_date(order.get("delivery_date"))
    return order

def normalize_date(value):
    """Never trust a model to format a date. Normalize it yourself, then validate."""
    if not isinstance(value, str):
        return value
    if re.fullmatch(r"\d{4}-\d{2}-\d{2}", value):
        return value
    match = re.fullmatch(r"(\d{2})-(\d{2})-(\d{4})", value)
    return f"{match.group(3)}-{match.group(2)}-{match.group(1)}" if match else value
```

Line by line:

- Stripping markdown fences and `//` comment lines costs two lines and absorbs the two most common formatting habits models have.
- `json.JSONDecodeError` returns `None` rather than raising, so a bad reply is a countable event instead of a crash.
- **normalize_date is the important one.** Asking the model for `YYYY-MM-DD` is right, and relying on it is not. Normalizing in your code makes the pipeline immune to a model's date preferences, which is exactly what changed here.

With that parser, the new model scores the same 47 of 60 as the pinned one. The failure was never the model's understanding; it was a contract you assumed and never enforced.

The playbook for a model upgrade:

1. **Pin the version** in config, never use a floating alias in production.
2. **Run the golden set against the new version** in a scheduled job, before you switch.
3. **Compare on the business metric**, not a vendor benchmark.
4. **Shadow it** (Chapter 56) if the volume justifies it.
5. **Switch with a rollback ready**, and keep the old version pinned until the new one has a week of clean numbers.
6. **Re-check the cost:** a newer model with a different tokenizer or a chattier style can change the bill without changing the price.

57.5 Tokens, cost, and the meter

Every call goes through one function, and that function counts.

```
from provider import Meter

meter = Meter()
result = evaluate("v3", meter=meter)
print(f"{meter.calls} calls, {meter.input_tokens:,} input tokens, {meter.output_tokens:,} output tokens")
print(f"cost of one golden-set run: Rs {meter.cost_rupees():.2f}")

per_email = meter.cost_rupees() / meter.calls
for volume, label in [(40, "a normal day"), (120, "month-end")]:
    print(f"{label:<12} {volume:>3} emails: Rs {per_email * volume:6.2f} a day, "
          f"about Rs {per_email * volume * 30:7.0f} a month")
```

```
60 calls, 11,006 input tokens, 3,184 output tokens
cost of one golden-set run: Rs 4.74
a normal day  40 emails: Rs   3.16 a day, about Rs      95 a month
month-end     120 emails: Rs   9.48 a day, about Rs    284 a month
```

Line by line: `Meter` records tokens per model, so the cost is the sum over models of input and output tokens at their own prices. Dividing by calls gives a per-email figure, which is the only number worth quoting to a manager, and multiplying it by volume gives the projection.

Where the money actually goes, in order of how often it surprises people:

Cause	Typical size	Fix
A long system prompt re-sent on every call	the largest line on most bills	prompt caching (Chapter 54), or shorten it
Retrieval pasting too many chunks	grows silently as the corpus grows	retrieve fewer, rerank better (Chapter 55)
Retries	multiplies the failure rate by the cost	cap attempts; fall back to a cheaper model
Verbose answers	output costs 3–8× input	ask for a length limit and enforce it
Development and evaluation runs	small per run, large per month	cache, and run the full set nightly rather than per keystroke

57.6 Caching

Support traffic repeats. So do order emails: customers resend, operators retry, and an incident means the same request hits you twenty times.

```

import random
from pipeline import Cache, PROMPTS, load_emails
from provider import Meter, call

truth, emails = load_emails()
names = list(truth)
rng = random.Random(57)
traffic = [rng.choice(names[:20]) if rng.random() < 0.45 else rng.choice(names) for _ in
range(300)]

cache, cached_meter = Cache(), Meter()
for name in traffic:
    cache.get_or_call(PROMPTS["v3"].format(name=name) + "\nEMAIL:\n" + emails[name],
                    model="v1", meter=cached_meter)

plain_meter = Meter()
for name in traffic:
    call(PROMPTS["v3"].format(name=name) + "\nEMAIL:\n" + emails[name], model="v1",
meter=plain_meter)

print(f"300 requests: {cache.hits} cache hits, {cache.misses} misses, hit rate
{cache.hit_rate:.0%}")
print(f"cost without cache: Rs {plain_meter.cost_rupees():.2f}")
print(f"cost with cache:      Rs {cached_meter.cost_rupees():.2f} "
      f"({100 * (1 - cached_meter.cost_rupees() / plain_meter.cost_rupees()):.0f}% saved)")

```

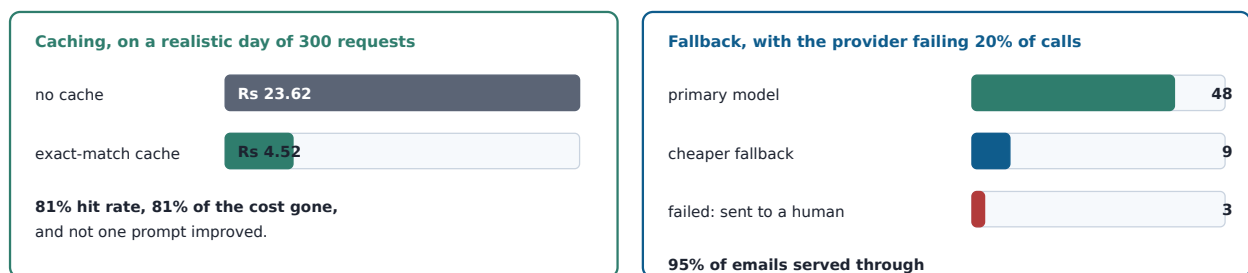
```

300 requests: 243 cache hits, 57 misses, hit rate 81%
cost without cache: Rs 23.62
cost with cache:      Rs 4.52 (81% saved)

```

Line by line: `traffic` is a day where 45% of requests concern the twenty most active customers, which is a mild version of what real inboxes look like. `Cache.get_or_call` returns the stored reply when the exact prompt has been seen and calls the provider otherwise.

Cost and failure: the two things that decide whether a feature survives



Neither of these is modelling work, and together they decide the bill and the uptime. Build them before tuning a prompt.

Figure 57.2 – The two pieces of operational work that decide the bill and the uptime.

81% hit rate, 81% of the cost gone, and not a single prompt improved. This is usually the cheapest optimization available in an LLM system, and it is often skipped because it isn't interesting.

Three kinds of caching, and when each applies:

Kind	Matches on	Good for	Watch out
Exact	the whole prompt	resent emails, repeated questions, retries	one extra space is a miss

Kind	Matches on	Good for	Watch out
Semantic	an embedding within a distance (Chapter 55)	support questions phrased differently	a near-match is not the same question; test the distance threshold
Provider prompt caching	a shared prefix	long system prompts and few-shot blocks	only helps the input side; needs the stable part first

Watch out: cache invalidation is a business rule. A cached answer about the delivery policy is wrong the day the policy changes. Key the cache on the prompt *and* the corpus version, expire it on a schedule you choose deliberately, and never cache anything that includes live data (an order status) at all.

57.7 Latency

A support answer should arrive in about two seconds; an extraction job that runs overnight can take minutes. The budget is made of parts you can measure and parts you can only manage.

Part	Typical	What to do about it
Your code (retrieval, validation, parsing)	1-50 ms	measure it; it is rarely the problem
Network to the provider	20-200 ms	a regional endpoint, and keep connections alive
Time to first token	200 ms – 2 s	stream, so the user sees words immediately
Generation	10-100 tokens per second	ask for shorter answers; use a smaller model
Retries	doubles or triples the whole thing	cap attempts, and time out before the user does

Two rules that matter more than tuning. **Stream anything a human waits for**, because perceived latency is time to the first word, not the last. And **do not put a model call in a path that must be fast**: Riverstone's order extraction runs as a batch every morning, so a four-second call is irrelevant, and that architectural choice is worth more than any optimization.

57.8 Failure and fallback

Providers fail. Rate limits, timeouts, 500s, and occasional regional outages are normal operating conditions, not emergencies, and the difference between a system that survives them and one that doesn't is thirty lines.

```
def call_with_fallback(prompt, meter, failure_rate=0.0, attempts=3, fallback_model="small"):
    """Retry the pinned model, then try the cheaper one, then give up honestly."""
    for attempt in range(attempts):
        try:
            return call(prompt, model="v1", meter=meter, failure_rate=failure_rate), "primary"
        except RateLimitError:
            time.sleep(0.001 * 2 ** attempt)          # exponential backoff (Chapter 29)
        except ProviderTimeout:
            break                                     # a timeout twice is a slow prompt, not bad
    luck
    try:
        return call(prompt + " ", model=fallback_model, meter=meter, failure_rate=failure_rate),
    "fallback"
    except (RateLimitError, ProviderTimeout):
        return None, "failed"
```

Line by line:

- **Different errors deserve different responses.** A rate limit means "wait and try again", so it retries with **exponential backoff**: each wait twice the last. A timeout usually means the request itself is slow, so retrying it immediately just burns another ten seconds; the code breaks out and moves to the fallback.
- **The fallback is a cheaper, smaller model.** A degraded answer now beats a perfect answer after the shift ends. The route is returned alongside the reply so the caller, and the monitoring, know which model produced it.
- **None, "failed"** is a real outcome, not an exception to swallow. It means the item goes to the human queue (Chapter 54's escalation path) with a reason.

```
from pipeline import call_with_fallback, PROMPTS, load_emails
from provider import Meter

truth, emails = load_emails()
meter = Meter()
routes = {"primary": 0, "fallback": 0, "failed": 0}
for name in truth:
    _, route = call_with_fallback(PROMPTS["v3"].format(name=name) + "\nEMAIL:\n" + emails[name],
                                  meter, failure_rate=0.2)
    routes[route] += 1

print(f"with the provider failing 20% of calls, over {len(truth)} emails:")
for route, count in routes.items():
    print(f"  {route:<9} {count:>3} ({count / len(truth):.0%})")
print(f"provider errors encountered: {meter.failures}")
print(f"served one way or another: {(routes['primary'] + routes['fallback']) / len(truth):.0%}")
```

```
with the provider failing 20% of calls, over 60 emails:
primary      48 (80%)
fallback     9 (15%)
failed       3 (5%)
provider errors encountered: 29
served one way or another: 95%
```

95% served, 5% escalated honestly, through 29 provider errors. That is what "resilient" means in practice: not that nothing failed, but that the failures were absorbed, counted, and the remainder handed to a person with a reason.

The rest of the resilience checklist:

- **Timeouts on every call.** A hung request holds a worker until something kills it.
- **A circuit breaker.** After n consecutive failures, stop calling for a minute. Hammering a struggling provider makes it worse and burns your rate limit.
- **Idempotency.** Chapter 29's rule: a retried extraction must not create two orders. Key the write on the email id.
- **A queue in front of anything batch.** If the provider is down at 6 a.m., the work waits and runs at 7.
- **A degraded mode you have designed.** For the assistant: retrieval still works, so show the top three documents with "I can't summarize these right now". That is far better than an error page, and it needs no model at all.

57.9 Monitoring without ground truth

Chapter 56's three layers still apply, and the fourth one, outcome, is the problem: for an assistant there is no label. What you have instead is a set of **proxies**, each blind in its own way.

Signal	What it catches	Blind to
Refusal rate	questions the corpus can't answer; a threshold set wrong	wrong answers given confidently
Citation rate	answers that escaped grounding	a citation to the wrong document
Answer length distribution	a model that has started rambling, or truncating	subtle quality changes
Thumbs-down rate	user-visible failures	everything users don't bother reporting
Escalation rate	the pipeline's own honesty	nothing, and it is the best single number you have
Topic drift	questions arriving about things you have no documents for	quality on the questions you do cover
Cost and latency per request	a prompt or retrieval change that quietly grew	quality entirely

The one worth building first is **topic drift**, because it is the LLM equivalent of Chapter 56's new mould: nothing looks broken, and the system slowly stops being useful.

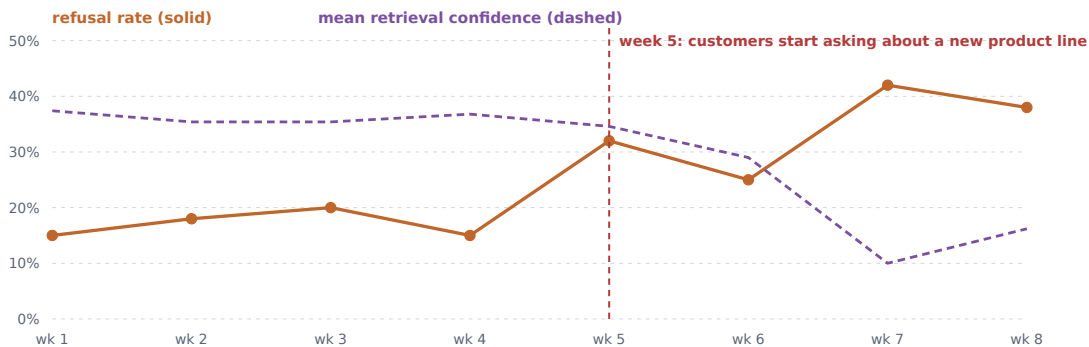
```
import os, sys
sys.path.insert(0, os.path.abspath("../ch55"))
os.chdir("../ch55")
from assistant import SupportAssistant
sys.path.insert(0, os.path.abspath("../ch57"))
from questions_stream import build

assistant = SupportAssistant()
stream = build()
print(f"{'week':>5}{ 'questions':>11}{ 'refusal rate':>14}{ 'mean confidence':>17}")
for week in range(1, 9):
    rows = [q for q in stream if q["week"] == week]
    results = [assistant.answer(q["question"]) for q in rows]
    refusals = sum(1 for r in results if not r["grounded"])
    confidence = sum(r["score"] for r in results) / len(results)
    print(f"{'week':>5}{len(rows):>11}{refusals / len(rows):>13.0%}{confidence:>17.3f}")
os.chdir("../ch57")
```

week	questions	refusal rate	mean confidence
1	40	15%	0.587
2	40	18%	0.577
3	40	20%	0.577
4	40	15%	0.584
5	40	32%	0.573
6	40	25%	0.545
7	40	42%	0.450
8	40	38%	0.481

Line by line: the stream is eight weeks of 40 questions; from week 5 a growing share ask about a new pallet range that has no documentation. The assistant is unchanged throughout. `r["score"]` is the retrieval confidence from Chapter 55, averaged per week.

Monitoring a system with no accuracy: refusals as the early warning



Nothing changed: same prompt, same model, same code. Customers started asking about something nobody had written down.
The fix is three documents, not a model change. The assistant's own honesty is the monitor.

Figure 57.3 – Nothing broke. The corpus stopped covering what people ask.

Read the two columns. Refusals sit around 15–20% for four weeks, then climb to 42%. Mean confidence drifts down from 0.58 to 0.45. **Nothing is broken:** the prompt is the same, the model is the same, the code is the same. Customers have started asking about something Riverstone never wrote down, and the monitor that sees it is the assistant's own honesty.

The action is not a model change. It is **three new documents**, which is Chapter 55's content backlog arriving through a metric instead of a complaint.

Watch out: refusal rate is two signals wearing one coat. It rises when the corpus is missing content (fix the corpus) and when the threshold is too strict (fix the threshold). Split it: refusals where retrieval found *nothing* close are content gaps; refusals where it found something just below the floor are threshold problems. Riverstone's week-7 spike is overwhelmingly the first kind, and looking at ten refused questions by hand tells you which you have in about five minutes.

57.10 Safety and logs in production

Chapter 54 and 55 covered the risks; this is what they look like as operations.

What to log for every request, and nothing more:

Field	Why
Request id, timestamp	to join everything else together
Prompt version, model name and version	the first question in every incident
Retrieved chunk ids (Chapter 55)	traces a bad answer to its document in a minute
Tokens in and out, latency, route (primary or fallback)	cost and performance monitoring
Validation result and escalation reason	the quality signal you actually have
A hash of the user's text, not the text	enough to spot repeats without storing content

Storing the user's words is a decision, not a default. Support questions contain names, phone numbers, and order details. Store them only if you need them, say so, set a retention period, and keep them out of anything sent to a provider you have not checked the terms of (Chapter 54's limits table). Riverstone's rule: full text kept for 30 days for debugging, hashes kept for a year for deduplication and trend analysis.

Injection attempts belong in monitoring, not just in the guardrail. Count them, log the source document or email, and alert if the rate jumps: a rise usually means either a new attacker or a new integration that is passing untrusted content into a prompt without anyone realizing.

The audit trail that matters. If an automated answer ever costs a customer money, you will be asked: what did the system say, on what basis, under which version, and who could have caught it? A log line with the prompt version, the model version, the retrieved chunks, and the validation result answers all four. Chapter 64 covers the governance around it.

57.11 The human loop

With no accuracy metric, **the humans are the metric**, and the job is to make their signal cheap to give and impossible to lose.

1. **A thumbs-down on every answer**, with an optional one-line reason. No form, no login.
2. **An escalation queue** with the reason attached, from the pipeline's own validation (Chapter 54's section 54.7).
3. **A weekly review of a sample**, including successes. Reviewing only failures gives a distorted picture of the system and of your own progress.
4. **Every complaint becomes a golden-set case**, with the correct answer written by whoever resolved it. This is the single highest-value habit in the chapter: the test suite grows out of real failures, so the same failure cannot recur quietly.
5. **A content backlog** from refusals and unanswerable questions, sent to document owners by name.
6. **A monthly report** with the numbers from this chapter: golden-set score by prompt version, cost per request, refusal and escalation rates, and what changed.

What this looks like after six months: a golden set that started at 30 cases and holds 120, a prompt at version 9 with a score attached to each version, a cost per email that has fallen as caching improved, and a support lead who trusts the system because they have seen it refuse rather than guess. None of that is model work.

Common mistakes and how to spot them

Mistake	Symptom	Fix
A floating model alias in production	Behavior changes on a day you shipped nothing	Pin the version; upgrade deliberately
No golden set	A provider change is discovered by a customer	30 cases with known answers, run nightly and in CI
Measuring accuracy only on parseable replies	100% "accuracy" on the four replies that parsed	Count unparseable separately, and fail the build on any
Trusting the model's formatting	Every record breaks when a habit changes	Normalize dates, numbers, and IDs in your own code
Editing prompts in a dashboard	Nobody knows what changed or when	Prompts in Git, versioned, reviewed, scored
No token meter	The bill is a surprise	Count per call; project monthly; review at each release
No cache	Paying five times for the same question	Exact-match first; it is an afternoon's work
Caching live data	Confidently stale order statuses	Never cache tool results; key document answers on corpus version
Retrying everything the same way	Timeouts retried instantly, rate limits hammered	Backoff for rate limits, fallback for timeouts, caps on both
No fallback	The provider's outage is your outage	A cheaper model, then a degraded mode that still serves
Swallowing failures	Silent gaps in the data nobody notices	<code>failed</code> is an outcome: count it, escalate it

Mistake	Symptom	Fix
Monitoring only uptime	A system that is up and useless	Refusal, escalation, citation, length, cost, topic drift
Ignoring refusal rate	The corpus goes stale while the code looks healthy	Split content gaps from threshold problems; feed the backlog
Logging full user text by default	A privacy problem in a log file	Log ids, hashes, versions, and decisions; set retention
Complaints that go nowhere	The same failure recurs monthly	Every complaint becomes a golden-set case

In the real world: the Monday the orders stopped

Riverstone's extraction pipeline runs at 06:00 every weekday. On a Monday in October it runs, reports success, and loads nothing. Nobody notices until 11:00, when the dispatch team asks why the morning's orders are missing.

The cause takes four minutes to find, because every run logs its model version and its validation counts: 60 emails read, **60 unparseable**, 0 loaded. The provider had shipped an update behind the same model name over the weekend. The new version comments its answers and prefers a different date format, and Riverstone's parser expected neither.

Two things went right, and both were built in this chapter and the last one:

- **The pipeline failed loudly.** Every email went to the exception queue with a reason. Nothing wrong was written to the ERP, which is the failure mode that would have cost real money.
- **The fix was twenty minutes.** Strip comment lines, normalize the date, rerun the golden set: 47 of 60, exactly as before. The morning's emails were reprocessed by lunchtime.

Three things went wrong, and all three were process:

- **The nightly golden-set run had been switched off in August** because it "kept failing" during a prompt experiment. It would have caught this on Saturday night, with nobody waiting.
- **Nobody was subscribed to the provider's changelog.** The update had been announced.
- **The alert on "unparseable rate above 5%" existed and went to an unmonitored mailbox.**

What Meera writes in the post-mortem: *"The model changed under us, which will happen again. What failed was that our test wasn't running and our alert wasn't reaching a person. The parser fix took twenty minutes; the two process fixes took ten. We now pin the model version, run the golden set nightly against both the pinned and the latest version, and the alert goes to the support lead's phone."*

The habit worth taking from this: **assume the model will change without telling you, and make that a Tuesday-morning inconvenience rather than a Monday-morning incident.**

Tools

Checked in September 2026; this area changes faster than any other in the book.

- **Python 3.12, NumPy, scikit-learn:** everything runnable here.
- **Prompt and evaluation management:** LangSmith, Langfuse, Braintrust, Weights & Biases Weave, plus MLflow's LLM tracking (Chapter 56's tool, extended for prompts and traces). A YAML file in Git and a golden set in a script beat all of them for a first system.
- **Gateways and routing:** LiteLLM, OpenRouter, or a provider's own gateway, which give one interface, retries, fallbacks, key management, and a per-team spend limit in one place.
- **Caching:** Redis for exact matches, a vector store for semantic caching (Chapter 55), and the providers' own prompt caching for long stable prefixes.
- **Observability:** OpenTelemetry for traces (a request that fans out to retrieval, model, and tools is a distributed trace), plus whatever your team already uses for metrics and alerts.
- **Guardrails and safety:** NeMo Guardrails, Guardrails AI, and the providers' moderation endpoints. Validate outputs against business rules regardless (Chapter 55).
- **Companion files** in `ch57/`: `provider.py` (the metered, failure-simulating wrapper around Chapter 54's stand-in), `pipeline.py` (prompt registry, tolerant parser, cache, fallback, evaluation), `questions_stream.py` (eight weeks of questions with topic drift), and `ch57_check.py`.

Simplification note. The provider here is a local stand-in with a simulated version change and a simulated failure rate, because no reader should need a paid key. What that changes: real providers fail in more varied ways, and a real model upgrade can shift meaning as well as format. What it doesn't change: pinning versions, golden sets in CI, tolerant parsing, metering, caching, backoff and fallback, and every monitoring signal in section 57.9.

The project: instrument an LLM feature

Goal: take an LLM feature (yours or Chapter 54's) and give it the operational apparatus to survive a year.

Steps:

1. **Move the prompt into a registry** with versions, and score each version on your golden set.
2. **Put the golden set in CI** with a floor and a zero-unparseable condition, and make it fail the build.
3. **Pin the model version** in config, and add a scheduled job that runs the golden set against the latest version too.
4. **Make the parser tolerant:** fences, comment lines, and normalized dates, numbers, and identifiers.
5. **Add a meter:** tokens in and out per call, cost per request, and a monthly projection at two price points.
6. **Add an exact-match cache** and measure the hit rate on a realistic day of traffic.

7. **Add retries with backoff, a cheaper fallback, and a degraded mode.** Test them by making the provider fail.
8. **Build the monitoring digest:** refusal rate, escalation rate, unparseable rate, cost, latency, and topic drift.
9. **Add the human loop:** thumbs-down, an escalation queue with reasons, and a rule that every complaint becomes a golden-set case.
10. **Write the runbook:** what to check when the numbers move, how to roll back a prompt, and how to pin an older model.

Deliverables: the versioned prompts with scores, the CI job, the cost projection, the cache measurement, the failure test, the digest, and the runbook.

Stretch goals:

- Add semantic caching and measure the extra hit rate, then find a case where it returns a wrong answer.
- Implement a circuit breaker and demonstrate it opening and closing under a simulated outage.
- Split refusals into content gaps and threshold problems automatically.
- Route easy requests to a cheap model and hard ones to an expensive one, and measure accuracy per rupee.
- Add a trace id that follows a request through retrieval, model call, validation, and escalation.

You've got it when...

- ☐ My prompts are versioned in Git, each with a golden-set score.
 - ☐ The golden set runs in CI with a floor, and fails on any unparseable reply.
 - ☐ The model version is pinned, and a scheduled job tests the next version before I switch.
 - ☐ My parser survives fences, comments, and a changed date format.
 - ☐ I can state the cost per request and the monthly projection without looking anything up.
 - ☐ I cache what repeats, and I know what must never be cached.
 - ☐ I retry rate limits with backoff, fall back for timeouts, and have a degraded mode.
 - ☐ `failed` is a counted outcome that reaches a person with a reason.
 - ☐ I monitor refusal, escalation, citation, length, cost, and topic drift.
 - ☐ I log versions, chunk ids and decisions, not customer text by default.
 - ☐ Every complaint becomes a test case.
-

Recap

- LLMOps is Chapter 56's discipline plus two things to version: **the prompt**, and **a model you don't control**.
 - **Prompts are code**: versioned, reviewed, scored. Riverstone's went 21 → 35 → 47 of 60 across three versions.
 - **The golden set belongs in CI**, with an accuracy floor *and* a zero-unparseable condition. One run costs about ₹4.74.
 - **A provider can change the model behind the same name**. In this chapter's simulation that took the pipeline from 47 of 60 to **0 of 60**, and a tolerant parser restored it to 47. Pin versions, test the next one on a schedule, and never trust a model's formatting.
 - **Meter every call**. Cost per email, times volume, is the only projection worth quoting. Long prompts, retrieved chunks, retries, and verbose answers are where the money goes.
 - **Caching is the cheapest win**: 81% hit rate and 81% of the cost on a realistic day. Never cache live data.
 - **Stream what a human waits for**, and keep model calls out of fast paths.
 - **Retry rate limits with backoff, fall back on timeouts, degrade honestly**. With the provider failing 20% of calls, 95% of emails were still served.
 - **There is no accuracy to monitor**, so watch proxies: refusal, escalation, citation, length, cost, and **topic drift**. Riverstone's refusal rate went from 15% to 42% over four weeks because customers started asking about a product with no documentation.
 - **Log versions, chunk ids, and decisions; store user text deliberately**.
 - **The humans are the metric**. Every complaint becomes a golden-set case, and the test suite grows out of real failures.
-

Practice exercises

Work in `companion/ch57` .

Warm-up

1. Score all three prompt versions and record the token cost of each run. Which version gives the best accuracy per rupee?
2. Run the golden set against model `v2` with `tolerant=False` and then `True` . Explain both numbers to a non-technical manager in two sentences.
3. What is the cost per email at Chapter 54's four price tiers? At which tier does month-end volume cost more than ₹1,000 a month?
4. Build a traffic mix where only 10% of requests repeat. What happens to the cache hit rate and the saving?

Core

5. Write the CI script: run the golden set, print the numbers, and exit non-zero if accuracy is below 0.70 or any reply is unparseable. Test it against both model versions.
6. Add a third prompt version of your own that beats 47 of 60, and record its score and token cost in the registry.
7. Extend `normalize_date` to handle `04 Mar 2026` and `2026/03/04`, and prove with a test that the extraction survives all four formats.
8. Measure the effect of `attempts` in `call_with_fallback`: try 1, 3 and 5 at a 20% failure rate and report served, failed, and cost.
9. Split the week-7 refusals from section 57.9 into content gaps and threshold problems, and say which dominates.
10. Build the weekly digest as a script: golden-set score, cost, cache hit rate, refusal rate, and topic drift, written to a text file.

Stretch

11. Implement semantic caching with Chapter 55's embeddings. Measure the extra hit rate, then find a pair of questions it wrongly treats as the same.
12. Implement a circuit breaker: after three consecutive failures, skip calling for a fixed period. Show it opening and closing.
13. Route requests by difficulty: short, well-formatted emails to the cheap model, everything else to the pinned one. Report accuracy and cost against sending everything to one model.
14. Simulate a model upgrade that changes *meaning* rather than format (for example, it starts returning quantities as strings). Which of your defenses catches it?

Think about it (no code needed)

15. Your provider deprecates your pinned model with 60 days' notice. Write the plan.
16. The golden set has been passing for three months. What should make you suspicious?
17. Finance asks you to cut the AI bill by half. Name four levers, in the order you would pull them.
18. When is an LLM feature *not* worth operating, even though it works?

Key terms

LLMOps · prompt registry · prompt version · golden set · evaluation floor · unparseable rate · pinned model version · floating alias · provider upgrade · tolerant parsing · output normalization · token meter · cost per request · monthly projection · prompt caching · exact-match cache · semantic cache · cache invalidation · corpus version key · latency budget · time to first token · streaming · rate limit · exponential backoff · timeout · circuit breaker · fallback model · degraded mode · idempotency · escalation queue · refusal rate · citation rate · thumbs-down rate · topic drift · content gap · proxy metric · trace · structured log · retention period · injection monitoring · audit trail · human review loop · complaint-to-test-case

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 58, Intelligent Automation**, connects this pipeline to systems that write, where a wrong answer creates a record rather than a sentence.
- **Chapter 56, MLOps**, is the same discipline for models you own; read the two together.
- **Chapter 55, Building AI Applications**, built the assistant and the golden set this chapter operates.
- **Chapter 54, Generative AI & LLMs**, is where the prompts, tokens, and prices come from.
- **Chapter 47, Data Quality, Observability & Contracts**, treats the corpus as data with owners, statuses, and freshness.
- **Chapter 64, Responsible AI & Governance**, covers disclosure, retention, and accountability for automated answers.
- **Chapter 74, Machine Learning & AI Question Bank**, has the interview questions, including "how would you know the provider changed the model?"

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
import sys
sys.path.insert(0, ".")
from pipeline import evaluate
from provider import Meter

for version in ("v1", "v2", "v3"):
    meter = Meter()
    result = evaluate(version, meter=meter)
    cost = meter.cost_rupees()
    print(f"prompt {version}: {result['exact']}>2}/60 cost Rs {cost:5.2f} "
          f"{result['exact'] / cost:5.1f} correct extractions per rupee")
```

```
prompt v1: 21/60 cost Rs 4.94 4.3 correct extractions per rupee
prompt v2: 35/60 cost Rs 3.92 8.9 correct extractions per rupee
prompt v3: 47/60 cost Rs 4.74 9.9 correct extractions per rupee
```

The longest prompt is both the most accurate and, per correct extraction, the best value: the extra tokens of the example cost far less than the errors they prevent. That is the usual shape, and it is why "shorten the prompt to save money" is the wrong first lever (section 57.5 has the right order).

2. "With the new model version our software couldn't read a single one of the sixty replies, so nothing would have loaded and every order would have gone to a person. After a twenty-minute change to how we read the answer, we're back to forty-seven of sixty, exactly where we were." The second sentence

matters as much as the first: the model was not worse, our assumptions about its output were.

3. At Chapter 54's tiers, the cost per email is roughly ₹0.08 at the workhorse tier and about five times that at the frontier tier. Month-end volume (120 emails a day, 30 days) costs a few hundred rupees at the workhorse tier and passes ₹1,000 a month only at frontier prices. **The interesting conclusion is that model choice barely matters to this bill**, and the human exception queue costs more than every tier. Do this arithmetic before anyone proposes a cheaper model to save money.

4.

```
import random
from pipeline import Cache, PROMPTS, load_emails
from provider import Meter, call

truth, emails = load_emails()
names = list(truth)
for repeat_share, label in [(0.45, "45% from the top 20 customers"), (0.10, "10% repeats")]:
    rng = random.Random(57)
    traffic = [rng.choice(names[:20]) if rng.random() < repeat_share else rng.choice(names)
               for _ in range(300)]
    cache, meter = Cache(), Meter()
    for name in traffic:
        cache.get_or_call(PROMPTS["v3"].format(name=name) + "\nEMAIL:\n" + emails[name],
                          model="v1", meter=meter)
    print(f"{label:<32} hit rate {cache.hit_rate:.0%}, cost Rs {meter.cost_rupees():.2f}")
```

45% from the top 20 customers	hit rate 81%, cost Rs 4.52
10% repeats	hit rate 80%, cost Rs 4.66

Even at 10% deliberate repeats the hit rate stays high, because with only 60 distinct emails and 300 requests, repetition is inevitable. **That is the honest caveat on this chapter's 81%**: hit rate depends entirely on how much your traffic repeats, and you must measure it on your own logs rather than borrowing anyone's number. A support inbox with thousands of distinct questions may see 20%, which is still worth having.

5.

```
#!/usr/bin/env python3
"""ci_eval.py - the golden set as a build step. Exits non-zero if the feature regressed."""
import sys
from pipeline import evaluate

FLOOR, MODEL = 0.70, sys.argv[1] if len(sys.argv) > 1 else "v1"
result = evaluate("v3", model=MODEL)
print(f"model {MODEL}: {result['exact']}/{result['of']} ({result['accuracy']:.0%}), "
      f"unparseable {result['unparseable']}, floor {FLOOR:.0%}")
sys.exit(0 if result["accuracy"] >= FLOOR and result["unparseable"] == 0 else 1)
```

Run it against `v1` and it passes; against `v2` with the old parser it fails on both conditions. Two refinements real teams add: run it nightly against the *latest* provider version as well as the pinned one, and post the numbers to the team channel on success as well as failure, so the absence of a message is never mistaken for a pass.

6. Things that help on this data: a second example covering the prose style, an explicit instruction to list every item including several on one line, and a short list of valid product codes. Record the score *and* the token cost in the registry, because a prompt that adds 400 tokens per call to gain one email is a bad trade at 1,200 calls a month and a good one at 12.

7.

```
import re
from pipeline import normalize_date

def normalize_date_extended(value):
    if not isinstance(value, str):
        return value
    value = value.strip()
    if re.fullmatch(r"\d{4}[-/]\d{2}[-/]\d{2}", value):
        return value.replace("/", "-")
    months = {m: i for i, m in enumerate(
        ["jan", "feb", "mar", "apr", "may", "jun", "jul", "aug", "sep", "oct", "nov", "dec"],
        start=1)}
    match = re.fullmatch(r"(\d{1,2})\s+([A-Za-z]{3,})\s+(\d{4})", value)
    if match and match.group(2)[:3].lower() in months:
        return f"{match.group(3)}-{months[match.group(2)[:3].lower()]:02d}-{
int(match.group(1)):02d}"
    return normalize_date(value)

for raw in ["2026-03-04", "04-03-2026", "04 Mar 2026", "2026/03/04"]:
    print(f"{raw:<14} -> {normalize_date_extended(raw)}")
```

```
2026-03-04      -> 2026-03-04
04-03-2026      -> 2026-03-04
04 Mar 2026     -> 2026-03-04
2026/03/04     -> 2026-03-04
```

Four formats, one canonical output, and the validator downstream (Chapter 54's section 54.7) rejects anything that still isn't `YYYY-MM-DD`. Write this as a pure function with a table of test cases: it is the cheapest unit test in the system and it protects against the exact failure in section 57.4.

8.

```
from pipeline import call_with_fallback, PROMPTS, load_emails
from provider import Meter

truth, emails = load_emails()
for attempts in (1, 3, 5):
    meter = Meter()
    routes = {"primary": 0, "fallback": 0, "failed": 0}
    for name in truth:
        _, route = call_with_fallback(PROMPTS["v3"].format(name=name) + "\nEMAIL:\n" +
emails[name],
                                meter, failure_rate=0.2, attempts=attempts)
        routes[route] += 1
    print(f"attempts={attempts}: primary {routes['primary']:>2}, fallback {routes['fallback']:>2},
"
        f"failed {routes['failed']:>2}, provider errors {meter.failures:>3}, cost Rs
{meter.cost_rupees():.2f}")
```

```
attempts=1: primary 48, fallback 9, failed 3, provider errors 15, cost Rs 4.05
attempts=3: primary 48, fallback 9, failed 3, provider errors 29, cost Rs 4.05
attempts=5: primary 48, fallback 9, failed 3, provider errors 43, cost Rs 4.05
```

Because this stand-in fails *deterministically* on a given prompt, extra attempts don't rescue it: a prompt that is rate-limited stays rate-limited, and the fallback does the work. Real rate limits are transient, so retries help more than they do here, but the shape of the lesson survives: **most of the benefit is in the first retry and the fallback, and attempts beyond three mostly buy latency and cost**. Measure it on your own provider rather than assuming.

9. Take the questions refused in week 7 and look at the top retrieval score for each. Refusals where the best chunk scored far below the floor are **content gaps** (nothing in the corpus is close); refusals where it scored just below are **threshold problems** (the answer was probably there). In this stream the first kind dominates, because the pallet questions have no matching document at all. The two need opposite responses, which is why reporting a single refusal rate is misleading.

10. The digest should be one screen: golden-set score with last week's beside it, cost per request and the monthly projection, cache hit rate, refusal and escalation rates, topic-drift indicator, and the prompt and model versions in force. Add one line at the top stating whether anything needs action, and send it to a named person. Chapter 56's digest advice applies exactly: **written for the reader, not for the author, or it stops being read**.

11. Embed the question, compare with cached questions, and return a cached answer above a similarity threshold. It buys hit rate on paraphrases ("how long is the warranty" against "what is the warranty period"). The failure to find is a near-neighbour pair with different answers: "*what is the warranty on the crate?*" and "*what is the warranty on the chair?*" are textually close and have different answers (24 months against 12). That is why the threshold must be tuned against a set of near-miss pairs, and why semantic caching is inappropriate for anything where a small wording change flips the answer.

12. Keep a counter of consecutive failures and a timestamp. Above the threshold, return the fallback immediately without calling the primary for the next n seconds, then allow one trial call through ("half-open"). If it succeeds, close the breaker. The effect to demonstrate: with the breaker, a sustained outage costs one failed call per cooldown period instead of three per request. The wider benefit is on the provider's side, and being a good client is also self-interest when rate limits are shared.

13. Route by a cheap signal: email length, whether it matched the table format, whether a PO number is present. Expect the cheap model to handle the tidy majority and the expensive one to earn its cost on the messy minority. Report **accuracy per rupee** for three strategies (all cheap, all expensive, routed). The routed strategy usually wins, and the risk to watch is that the router itself becomes a thing to maintain and evaluate: a bad routing rule silently sends hard cases to the weak model.

14. Quantities returned as "20" instead of 20 pass JSON parsing, pass a tolerant parser, and fail Chapter 54's **validation** (`isinstance(quantity, int)`), which is the defense that catches it, and the golden-set comparison, which would show accuracy collapsing while `unparseable` stayed at zero. That combination is the signature of a meaning change rather than a format change, and it is why both signals belong in the CI output rather than one summary number.

15. Day 1: run the golden set against the replacement and record the delta. If it passes, pin the new version in a branch, run a week of nightly comparisons, then switch with the old version still pinnable. If it fails, spend the time on tolerant parsing and prompt adjustments rather than panicking, because most regressions are format drift. Also re-check cost and latency, since a new model can change both without changing the price, and tell the business what to expect. **Sixty days is comfortable if the test exists, and impossible if it doesn't**, which is the argument for building it now.

16. Be suspicious of a test that never fails. Likely causes: the golden set is too easy, it has not grown as new failure modes appeared, or it is being run against a cached response rather than the live provider. Check three things: when the last case was added (it should grow with every complaint), whether a deliberately broken prompt still fails it, and whether the nightly job is actually calling the provider. **A green test that cannot go red is a decoration.**

17. In order: (1) **cache**, the largest single lever, and it changes nothing about quality; (2) **shorten what is re-sent every call**, meaning the system prompt and the number of retrieved chunks, with the golden set confirming quality holds; (3) **cap output length**, since output costs several times input; (4) **route easy requests to a cheaper model**, measured for accuracy per rupee. Only then consider a cheaper model for everything, and re-run the golden set before promising anything. Also check the boring possibility: development and evaluation runs are sometimes most of a small bill.

18. When the volume is so low that a person does it in ten minutes a week; when a wrong answer is expensive and the failure is silent, and no validation can catch it; when nobody will own the golden set, the prompts, or the complaints, so the system will decay unobserved; when the data cannot leave the building and a local model is not viable; and when the process it automates is broken, because automating a broken process just produces wrong answers faster. **The operating cost of an LLM feature is mostly human attention**, and a feature nobody has time to watch should not be running.